

Software Design Patterns

1. What are Software Design Patterns?

Design patterns

- Represent the best practices followed by experienced software developers.
- Targeted to solve the problems of object creation & integration.
- Design Patterns act as templates for real world problems.
- In 1994, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides published a book titled.
- First part: Pros & Cons of object oriented programming -9
- Second part: 23 design Pattern.
- "Design Patterns - Elements of Reusable Object - Oriented Software" (Gang of Four)
 - Creational: object creation & initialization
 - Structural: Decoupling interface & implementations
 - Behavioural: Communication between classes & objects

2. What are Creational Design Patterns?

Creational Design Patterns

Software Design

Patterns

- Deal with object creation mechanism.
- Create objects based on use case.
- Reduce the complexity of object creation.
- Define the best possible way to reuse it.

→ Abstract Factory → Creates an instance of several families of classes.

→ Builder → builds complex object from scratch.

→ Factory → Creates an instance of several derived classes.

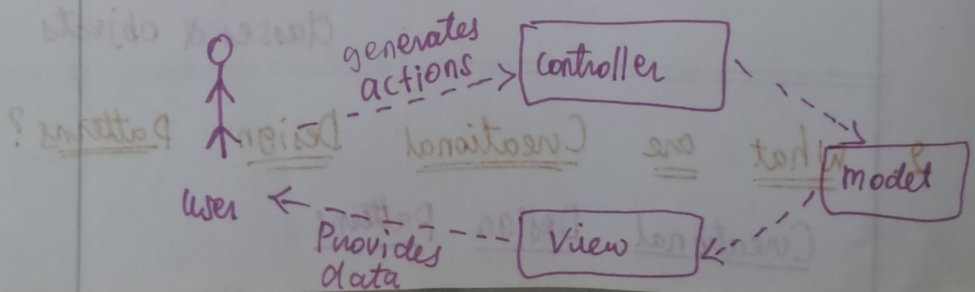
→ Object Pool → recycle the objects

→ Prototype → Copy or clone

→ Singleton → only one instance can be created.

3. Model View Controller (MVC) :-

Model / View / Controller



→ Model :

→ holds data & manipulates it.

→ Example : student object holds name, marks & etc.

→ View :

→ Screen presentation.

→ Example : pie chart, bar chart, and etc.

→ Controller :

→ act as a mediator between the Model & View.

→ defines the way the user interface reacts to the user.

Main aim of MVC is Decouple the things (contracts).

4. What is Singleton Design Pattern?

Singleton Design Pattern

→ Only one object creation.

→ If we want the common things from the object then use Singleton Design pattern.

Ex : Student lenin = new Student (1,100);

Student chenchu = new Student (2,100)

Advantages:

→ Saves memory

→ Performance improvement

(Every object has specific thing)

So, no Singleton

Implementation Rules:

→ Private Constructors

Real world Examples:

1) → Java course by Rakesh

→ one object Rakesh

→ All the viewers use

2) Runtime class

to get information from that Rakesh object.

5. Singleton Design Pattern Implementation in Java :-

→ Open Netbeans and Create new Java Project

→ Then Name Singleton.java (file).

→ Program :-

Package learn java easily . design Patterns ;

public class Singleton

{

private static Singleton singleton;

/* Create private constructors

private Singleton ()

{

}

/* Create a static method to get instance.

public static Singleton getInstance ()

{

if (singleton == null)

{

singleton = new Singleton ();

}

return singleton;

}

public void displayMessage ()

{

System.out.println ("I have called
using Singleton object");

}

}

→ Open new file to create SingletonMain.java.

→ Program:-

```
package learnjavaeasy.designPatterns;  
  
public class SingletonMain  
{  
    public static void main (String[] args)  
    {  
        // Not possible to create an object using  
        // new keyword because the constructor  
        // is private  
        Singleton singleton = new Singleton();  
  
        Singleton singleton1 = Singleton.getInstance();  
        Singleton singleton2 = Singleton.getInstance();  
        singleton1.displayMessage();  
        singleton2.displayMessage();  
    }  
}
```

Output :-

I have called using singleton object
I have called using Singleton Object

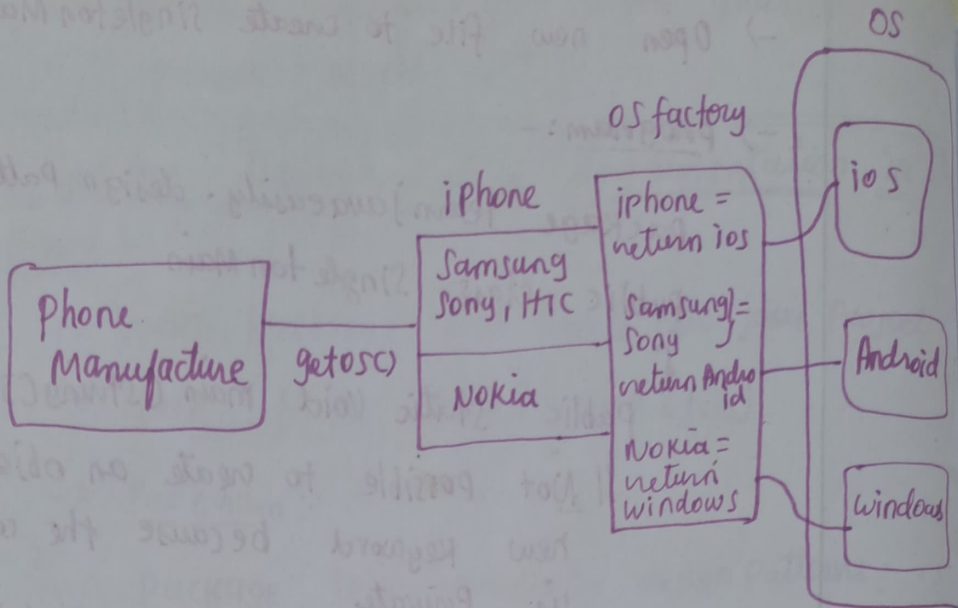
6. What is Factory Design Pattern?

Factory Design Pattern

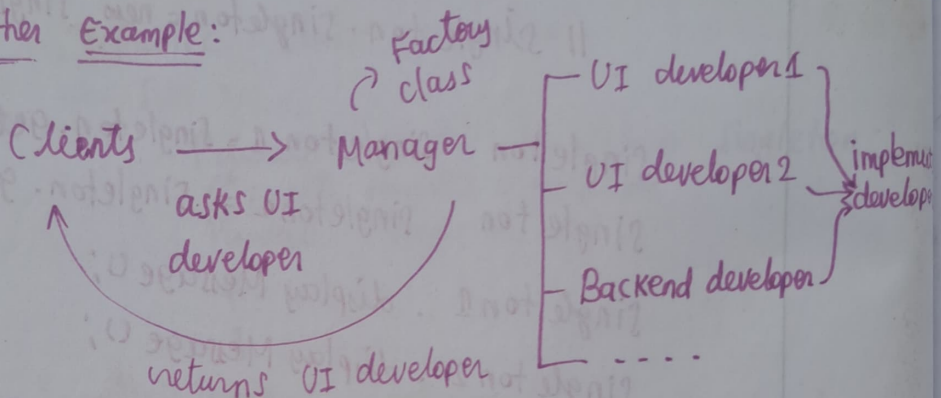
→ falls under Creational design Pattern.

→ Create object without exposing the creation logic to the client.

Example :-



Other Example:



7. Factory Design Pattern implementation in Java:

→ Open Netbeans Create java Project.

→ Open Food.java, Veg Food.java, Nonveg Food.java, Foodsewer.java, Customer.java.

→ Program :- learn java easily. design Patterns;

package

import java.util.*;

public abstract class Food

{

protected double bill person;

protected Set<String> items;

public Food (double bill PerPerson)

{

this. bill PerPerson = bill Per Person;

this. items = new HashSet<>();

}

public double get Bill ()

{

```

        return bill Per Person;
    }
    public Set <String> getItems() {
        return items;
    }
}

```

→ Output :-

Customer 1

Items: [Vada, Dosa, Idly]

Bill: 100.0

Customer 2

Items: [Biryani, Chicken 65]

Bill: 120.0

→ Open Customer.java

→ Program :-

```

package learnjavaeasily.designPatterns;
public class Customer

```

```

{
    public static void main (String [] args)

```

```

    {
        Food CustomerFood = FoodSewer.getFood("veg");

```

```

        System.out.println ("Customer 1");

```

```

        System.out.println ("Items: " + CustomerFood.getItems().toString());

```

```

        System.out.println ("Bill " + CustomerFood.getBill());

```

```

        Food Customer2Food = FoodSewer.getFood ("non veg");

```

```

        System.out.println ("Customer 2");

```

```

        System.out.println ("Items : " + Customer2Food.getItems().toString());

```

```

        System.out.println ("Bill : " + Customer2Food.getBill());

```

```

    }

```

```

}

```


→ Open Foodsewer.java

→ Program:-

Package : learn Java easily . design Patterns ;

public class Foodsewer

{

public static Food getFood (String
Food Type)

{

if (foodType . equals ("veg"))

{

return new Veg Food ();

}

else if (foodType . equals ("non veg"))

{

return new NonVeg Food ();

}

else

{

System . out . println ("we are not
Sewing "+ foodType);

return null;

}

}

}

→ This code mainly important

→ Open NonVegFood.java

→ Program:-

Package learn java easily . design patterns;

Public Class NonVegFood extends Food
{

Public NonVegFood()

{

Super (120);

items . add ("Biryani");

items . add ("Chicken 65");

→ Open VegFood.java

→ Program:-

Package learn java easily . design patterns;

Public Class VegFood extends Food

{

Public VegFood()

{

Super (100);

items . add ("Idly");

items . add ("Dosa");

items . add ("Vada");

}

Q. What is Abstract Factory Design Pattern?

Abstract Factory Design :-

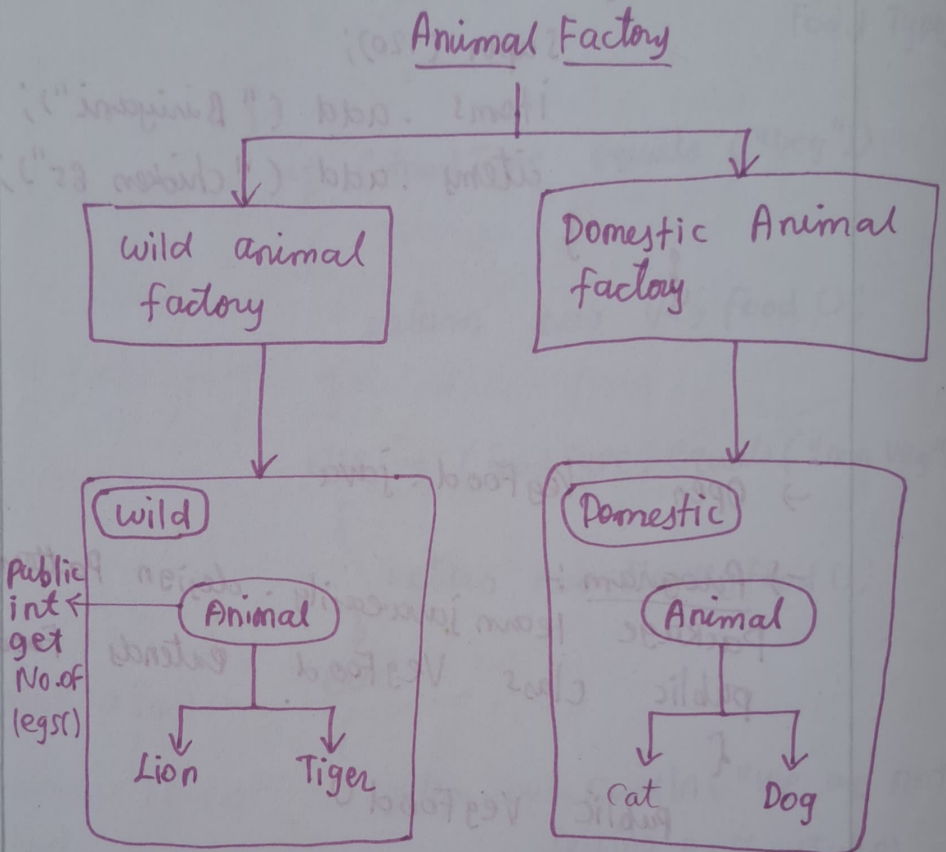
→ Super factory which creates other factories

→ Provides interface for ~~creating~~ ^{create} objects

(related or dependent) without specifying the concrete classes.

→ Abstract means hiding, Factory - Provides object based on type.

Real-time Example



Java Library Example:-

→ Reading XML files using DOM parser.

// Example

Document Builder Factory docBuilderFactory = Document Builder
Factory.getInstance();

Document Builder factory = docBuilderFactory.newDocumentBuilder();

Document doc = factory.parse(etc);

9. Abstract Factory Design Pattern Java Program:-

→ Open Netbeans

→ Then create java file (ExampleMain.java, Course.java, ProgrammingCourse.java, NonProgrammingCourse.java, CourseFactory.java, SourceFactory.java, offline.java, online.java, SourceCourseFactory.java, FactoryCreator.java, SourceFactory.java).

→ Open Example main.java

→ Program:-

```
package learnjavaeasily.designPatterns.abstract
factory;

public class ExampleMain
{
    public static void main (String [] args)
    {
        Source Course Factory course = Factory Creator .
            get Source Course Factory ("Course");

        System.out.println (Course.get course ("Program
            -ming").get Course Name ());

        Source Course Factory Source = Factory Creator .
            get Source Course Factory ("Source");

        System.out.println (Source.get Source ("online").
            get Source Name ());
    }
}
```

→ Open course.java file

→ Program:-

```
package learnjavaeasily.designPatterns.abstract
factory;
```

Abstract Course Interface

```
{ public String get Course Name();  
}
```

→ Open Programming Course. java

→ Program :-

```
package learnjavaeasily.designpatterns.abstract  
factory;
```

```
public class Programming Course implements  
Course
```

```
@Override
```

```
public String get Course Name()
```

```
{
```

```
return "Java";
```

```
}
```

```
}
```

→ Open Non Programming Course. java .

→ Program :-

```
package learnjavaeasily.designpatterns.abstract  
factory;
```

```
public class Non Programming Course implements  
Course
```

```
{
```

```
@Override
```

```
public String get Course Name()
```

```
{
```

```
return "Dsp";
```

```
}
```

```
}
```

→ Open CourseFactory.java.

→ Program :-

Package | learnjavaeasily. designPatterns. abstract;
factory;

Public class CourseFactory extends Source
Course Factory

```
{  
    @Override  
    public Source getSource (String SourceType)  
    {  
        return null;  
    }  
    @Override  
    public Course getCourse (String CourseType)  
    {  
        if (courseType.equals IgnoreCase ("programming"))  
        {  
            return new ProgrammingCourse ();  
        }  
        else if (courseType.equals Ignore Case (" non  
            programming"))  
        {  
            return new Non Programming Course ();  
        }  
        else {  
            return null;  
        }  
    }  
}
```

→ Open : Source.java

→ Program :-

package learnjavaeasily. designPatterns. abstract
factory;

```
public interface Source  
{  
    public String getSourceName();  
}
```

→ Open Offline.java

→ Program:-

```
package learnjavaeasily. designPatterns. abstract  
factory;
```

```
public class offline implements Source  
{  
    @Override  
    public String getSourceName()  
    {  
        return "Books";  
    }  
}
```

→ Open online.java

→ Program:-

```
package learnjavaeasily. designPatterns.  
abstract factory;
```

```
public class online implements Source  
{  
    @Override  
    public String getSourceName()  
    {  
        return "YouTube";  
    }  
}
```

→ Open Source Course Factory . java

→ Program :-

Package learnjavaeasily . design patterns . abst
-nact factory;

```
Public abstract class Source Course Factory
{
    public abstract Source getSource (String
                                   Source Type);

    public abstract Course getCourse (String
                                   CourseType);
}
```

→ open Factory Creator . java

→ Program :-

Package learnjavaeasily . design patterns . abstract
factory;

```
Public class FactoryCreator
{
    public static Source Course Factory getSource
    Factory (String type)
    {
        if (type . equals ("course"))
        {
            return new CourseFactory ();
        }
        else if (type . equals ("source"))
        {
            return new Source Factory ();
        }
        else
        {
            return null;
        }
    }
}
```

-> Open Source Factory.java

-> Program :-

Package learn java easily . designPatterns. abstract
- (+factory;

Public class Source Factory extends Source
Course Factory

{

@Override

public Source getSource (String SourceType)

{

if (SourceType. equals Ignore Case('online'

return new online();

}

else if (SourceType. equals Ignore Case
("offline"))

{

return new offline();

}

else

{

return null;

}

}

@Override

public Course getCourse (String
Course Type)


```

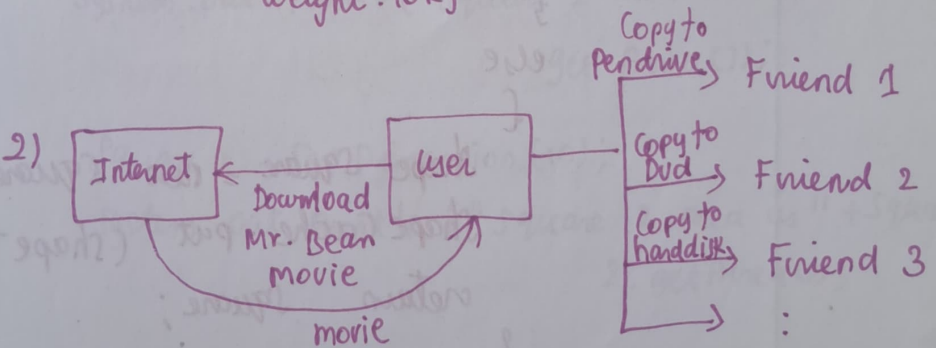
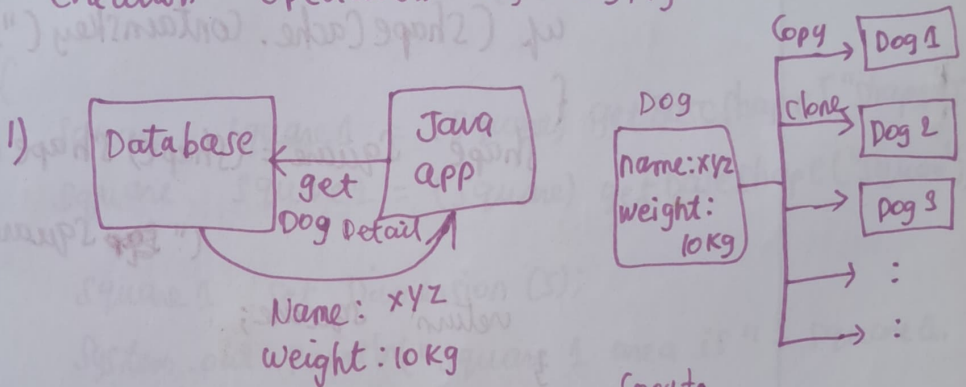
    }
    return null;
}
}

```

10. What is Prototype Design Pattern?

Prototype Design Pattern

- > Creational design pattern
- > Create new objects by cloning or copying
- > This pattern is used when the object creation operation is costly.



11. Prototype Design Pattern Program in Java:

- > Open Netbeans
- > Open prototype main.java. [create file prototype main.java, Rectangle.java, Shape.java, Square.java]
- > Program:-

package learnjavaeasy.designPatterns.prototype;

```

import java.util. HashMap;
import java.util. Map;

```

Public Class Prototype Main

{

Public Static Map <String, Shape> shapeCache =
new HashMap<>();

Private Static Shape getBaseShape (String
shapeType) throws CloneNotSupportedException

{

if (shapeType.equalsIgnoreCase("square"))

{

if (shapeCache.containsKey("square"))

{

Shape square = (Shape) shapeCache.get
("square").clone();

return square;

}

else

{

Shape square = new Square();

shapeCache.put (shapeType, square);

return square;

}

}

else if (shapeType.equalsIgnoreCase("rectangle"))

{

if (shapeCache.containsKey("rectangle"))

{

Shape rectangle = (Shape) shapeCache.
get ("rectangle").clone();

return rectangle;

}

```

    else
    {
        Rectangle rectangle = new Rectangle ();
        ShapeCache.put (ShapeType, rectangle);
        return rectangle;
    }
}

```

```

return null;
}

```

```

public static void main (String [] args) throws
    (Clone Not Supported Exception

```

```

{
    Square square1 = (Square) getBaseShape ("square");
    Square square2 = (Square) getBaseShape ("square");

```

```

    square1 . Set Dimension (5);

```

```

    System.out.println ("square 1 area is " + square1.
        get Area());

```

```

    square2 . Set Dimension (10);

```

```

    System.out.println ("square 2 area is " + square
        2 . getArea());

```

```

    Rectangle rectangle1 = (Rectangle) get Base
        Shape ("rectangle");

```

```

    Rectangle = rectangle2 = (Rectangle) get Base
        Shape ("rectangle");

```

```

    rectangle1 . set length (2);

```

```

    rectangle1 . Set Breadth (1);

```

```

    System.out.println ("Rectangle 1 area is " +
        rectangle . getArea());

```

```

    rectangle2 . set length (4);

```



```
rectangle.setBreadth(2);
```

```
System.out.println("Rectangle 2 area is",
```

```
rectangle2.getArea());
```

```
}  
}
```

→ Output → Square 1 area is 25.0

Square 2 area is 100.0

Rectangle 1 area is 2.0

Rectangle 2 area is 8.0

→ Open Rectangle.java

→ Program :-

```
public class Rectangle extends Shape  
implements Cloneable
```

```
{  
    private double length;  
    private double breadth;
```

```
    public Rectangle()
```

```
    {  
        super("Rectangle");
```

```
    }
```

```
    public void setLength(int length)
```

```
    {
```

```
        this.length = length;
```

```
    }
```

```
    public void setBreadth(int breadth)
```

```
    {
```

```
        this.breadth = breadth;
```

```
    }
```

```
    public double getArea()
```

```
    {
```

return length * breadth;

}

@Override

public Object clone() throws CloneNotSupportedException

{

Rectangle cloneRectangle = new Rectangle();
return cloneRectangle;

}

}

→ Open Shape.java

→ Program :

Package learnjavaeasily.designpatterns.prototype;

public class Shape implements Cloneable
{

protected String shapeName;
protected String description;
protected double area;

public Shape(String shapeName)

{
this.shapeName = shapeName;
this.description = "This class is about " +
shapeName;

}

public String getShapeName()

{
return this.shapeName;

}

public String getDescription()

{

return this.description;

}

@Override

public Object clone() throws CloneNotSupportedException

{

Shape clonedShape = new Shape(this.shapeName);

return clonedShape;

}

}

→ Open Square.java

→ Program:-

Package learnjavaeasily.designpatterns.proto
type;

public class Square extends Shape
{

private double dim;

public Square()

{
super("Square");
}

public void setDimension(double dim)

{
this.dim = dim;

public double getArea()

{
return dim * dim;

@ Override

Public Object clone() throws CloneNotSupportedException

```
{  
    Square clonedSquare = new Square();  
    return clonedSquare;  
}
```

12. What is Object Pool Design pattern?

Object Pool Design Pattern

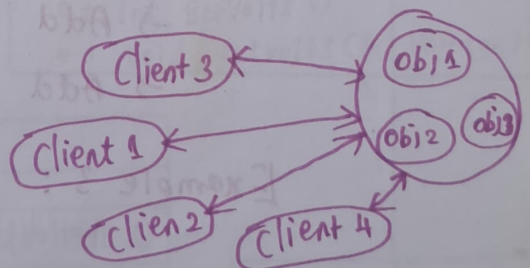
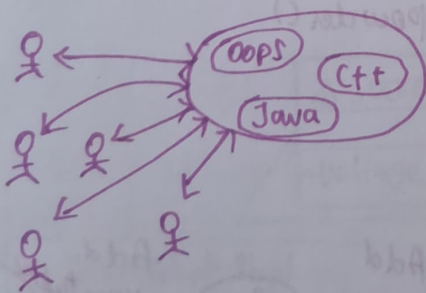
→ Creational design pattern

→ reuse the objects that are expensive to create.

Real-time Example :

Reusable Pool Size = 3

Library



Database Connection Pool

→ opening too many database connections will affect performance.

→ Creating n connections is expensive.

→ Objects in pool have a lifecycle creation, validation and destroy.

Advantages:

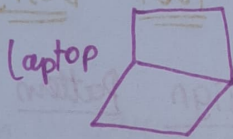
- It boosts the performance
- It saves memory.
- It reuse the objects so it saves resources

13. What is Builder Design Pattern?

Builder Design Pattern

- Creational Design Pattern
- builds complex object using simple objects

Example 1:



- plug the hard disk
- plug the memory
- set LCD screen
-

Example 2:

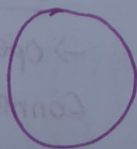
Create Beverage

Coffee, Boost, Horlicks, ...

- Add water(), Add milk()
- Add Sugar()
- Add powder()

Example 3:

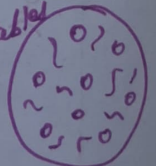
Pizza:



Add
cheese
⇒



Add
vegetables
⇒



plain pizza

cheese
plain pizza

Pizza

14. What is Adapter Design Pattern?

Adapter Design Pattern

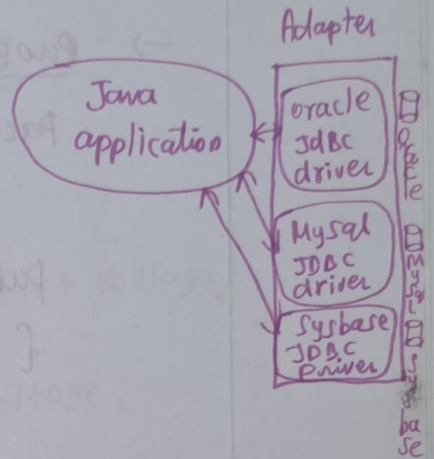
→ Structural Design Pattern

→ bridge between two incompatible interfaces

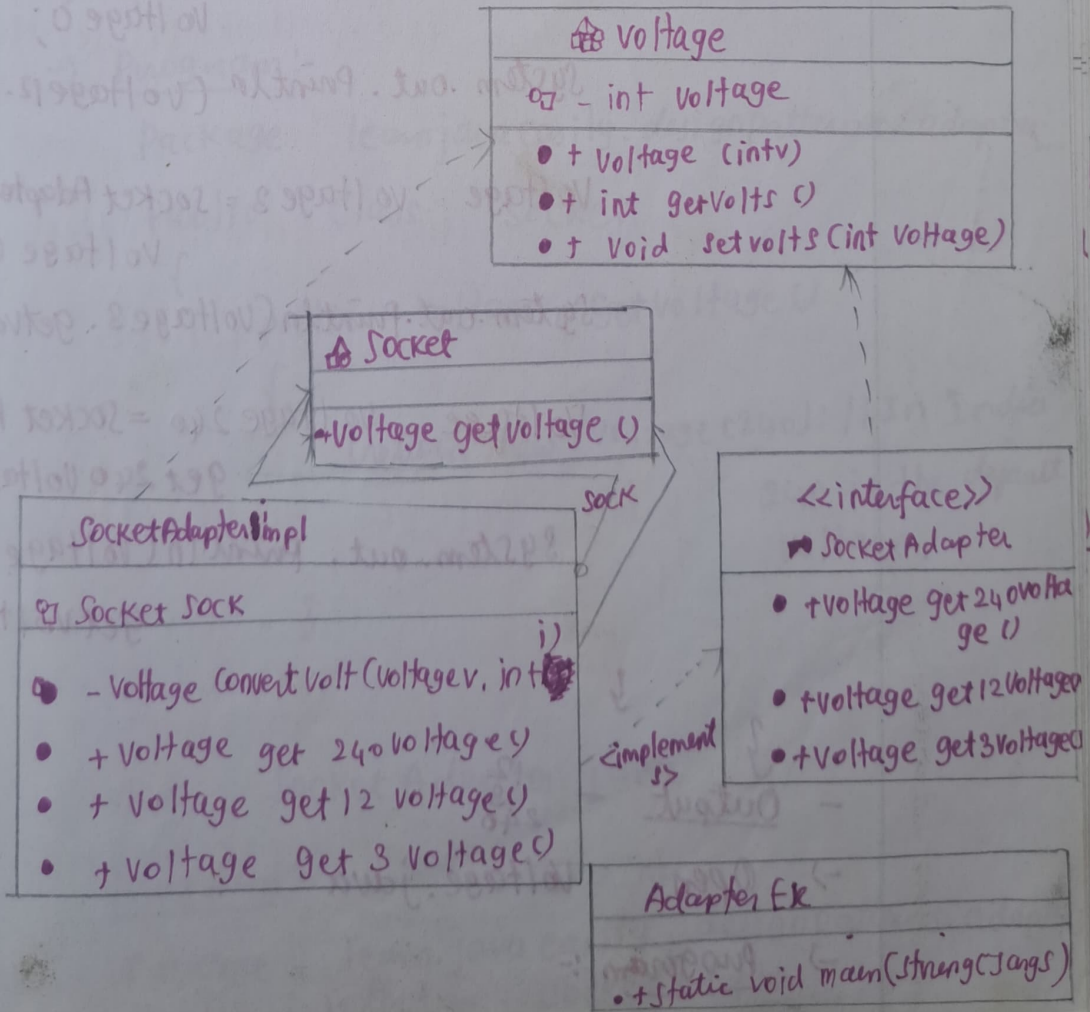
Real-life example

- 1) Mac USB C - USB
- 2) memory Card Reader
- 3) phone Adapter 220 v to 9v

Technical Example



15. Adapter Design Pattern Java Program:-



• Mobile battery needs 3 Volts to charge but the normal socket produces 240V in india. So the mobile charger works as an adapter between mobile charging socket and the wall.

→ Open netbeans then create Project.

→ Open AdapterEx.java [AdapterEx.java, Voltage.java, Socket.java, SocketAdapter.java, SocketAdapterImpl.java]

→ Program:-

Package learn java easily . design patterns . adapter;

Public class AdapterEx

{

Public static void main (String[] args)

{

SocketAdapter socketAdapter = new SocketAdapterImpl();

Voltage voltage12 = socketAdapter.getVoltage();

System.out.println (voltage12.getVolts());

Voltage voltage3 = socketAdapter.getVoltage();

System.out.println (voltage3.getVolts());

Voltage voltage240 = socketAdapter.getVoltage();

System.out.println (voltage240.getVolts());

}

- Output -

12
3
240

→ Open Voltage.java

→ Program:-

Package learnjavaeasyly . designpatterns . adapter;

Public class Voltage

{

Private int voltage;

Public Voltage (int v)

{

this . voltage = v;

}

public int getVolts ()

{

return voltage ;

}

public void setVolts (int voltage)

{

this . voltage = voltage ;

}

}

→ Open Socket . java

→ Program:-

Package learnjavaeasyly . designpatterns . adapter;

Public class Socket

{

public Voltage getVoltage ()

{

return new Voltage (240); // In India

240 is the default
voltage.

}

}

→ Open Socket Adapter . java

→ Program:-

Package

learnjavaeasyly . designpatterns . adapter;

Public

interface Socket Adapter

```

    public Voltage get 240 Voltage ();
    public Voltage get 12 voltage ();
    public Voltage get 3 V Voltage ();
}

```

→ Open Socket Adapter impl. java

→ Program :-

Package learn java easily . design patterns . SocketAdapterImpl;

Public class SocketAdapterImpl extends Socket implements SocketAdapter

{

//using Composition for adapter pattern

Private Socket sock = new Socket ();

Private Voltage convert Volt (Voltage v, int

{

return new Voltage (v.getVolts () / 11);

}

@Override

Public Voltage get 240 Voltage ()

{

return sock. getVoltage ();

}

@Override

Public Voltage get 120 Voltage ()

{

Voltage v = sock. getVoltage ();

return convert Volt (v, 20);

}

@Override

Public Voltage get 3V Voltage ()

{

Voltage v = sock. getVoltage ();

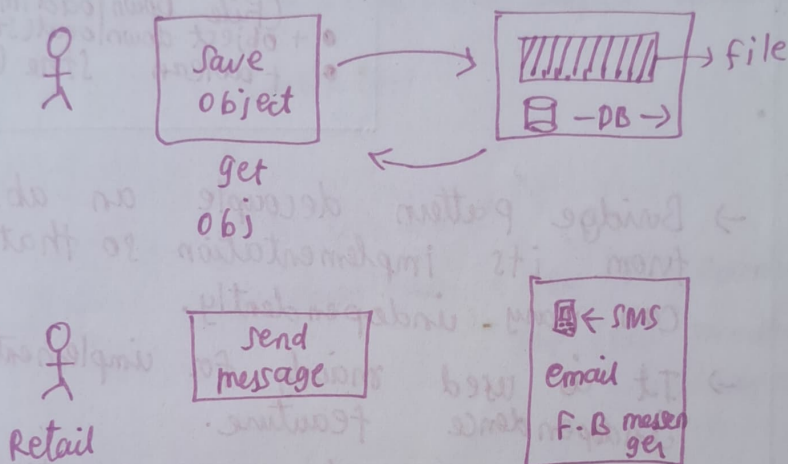
return Convert Volt (v, ao);

16. Bridge Design Pattern real-time examples:

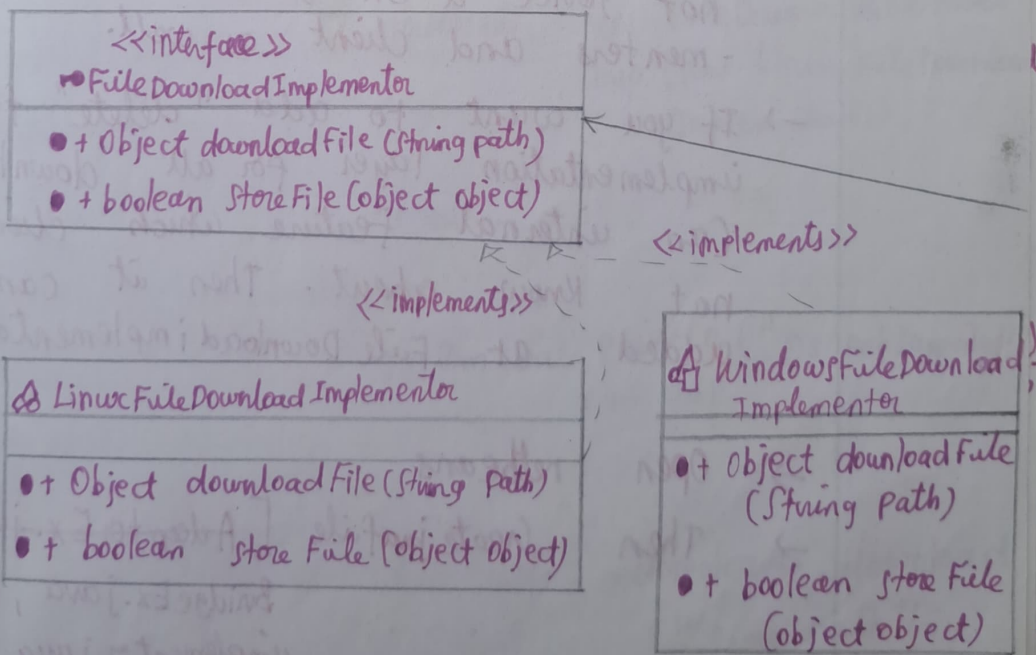
→ Bridge Design Pattern - 

→ Abstraction & Implementation

→ then fall on Structural design pattern



17. Bridge Design Pattern Java Implementation:-



Bridge Ex

• + static void main (String[] args)

<< interface >>

File DownloaderInterface

- + object download (String path)

- + boolean store (object object)

<< implements >>

File Downloader AbstractImpl

- + File Downloader AbstractImpl
- (File Download implementer provide)
- + object download (String path)
- + boolean store (object object)

→ Bridge pattern decouple an abstraction from its implementation so that the two can vary independently.

→ It is used mainly for implementing platform independence feature.

→ If you want to add one more capability "delete" at abstraction layer. It must not force a change in existing implementers and client as well.

→ If you want to add "delete" feature at implementation layer for all downloaders (an internal feature which client should not know about. Then it can be added at File Download implementor.

→ Open netbeans

→ Then create java file [AdapterEx.java,

BridgeEx.java, File Downloader

implementor.java, File Downloader

AbstractImpl.java, File Downloader

-loaderinterface.java, Linux

File download implementer for

Windows File Download

java

→ Open BridgeEx.java

→ Program:

Package learn.javaeasily.designpatterns.bridge;

public class BridgeEx

{
public static void main (String[] args)

{
String os = "Linux";

File Downloader^{er} Interface downloader = null;

Switch (os)

{
Case "Windows":

downloader = new File Downloader Abstract
Imp (new Windows File Down
-load Implementor ());

break;

Case "Linux":

downloader = new File Downloader Abstract
Imp (new Linux File Download
Implementor ());

break;

default :

System.out.println ("Os not supported!!");

}

Object File Content = downloader.download
("some path");

downloader.store (file content);

}

}

→ Open File Download Implementor.java

→ Program :-

package learn java easily . design patterns . bridge

public interface File Download Implementor

{

public object download File (String path);

public boolean store File (object object);

}

→ Open File Downloader Abstract Impl.java.

→ Program :-

package learn java easily . design patterns . bridge

public class File Downloader Abstract Impl

implements File Downloader Interface

{

private File Download Implementor file down

loader = null;

public File Downloader Abstract Impl (File
Downloader Implementor provider)

{

super();

this . file Downloader = provider;

}

@Override

public object download (String path)

{

return file Downloader . download

File (path);

}

@ override

```
public boolean store (Object object)
```

```
{
```

```
    return fileDownloader.storeFile(object);
```

```
}
```

```
}
```

→ Open FileDownloaderInterface.java

→ Program :-

Package learnjavaeasily . designPatterns . bridge;

```
public Interface FileDownloaderInterface
```

```
{
```

```
    public Object download (String path);
```

```
    public boolean store (Object object);
```

```
}
```

→ Open LinuxFileDownloadImplementor.java

→ Program :-

Package learnjavaeasily . designPatterns . bridge;

```
public Class LinuxFileDownloadImplementor Implementor  
    implements FileDownloadImplementor
```

```
{
```

@ override

```
    public Object downloadFile (String path){
```

```
        return new Object ();
```

```
}
```

@ override

```
    public boolean storeFile (Object object){
```

```
        System.out.println("File stored successfully  
        in LINUX machine");
```

```
        return true;
```

→ Open windows File Download implementor.java

→ Program:-

```
package learnjavaeasily.designpatterns.bridge;

public class windowsFileDownloadImplementor
    implements FileDownload
        Implementor
```

```
@Override
```

```
public Object downloadFile (String path){
    return new Object ();
}
```

```
@Override
```

```
public boolean storeFile (Object Object){
    System.out.println ("File stored successfully
        in WINDOWS machine");
    return true;
}
```

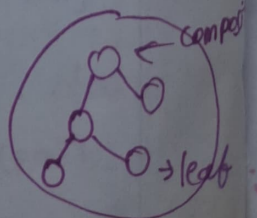
18. What is Composite Design pattern?

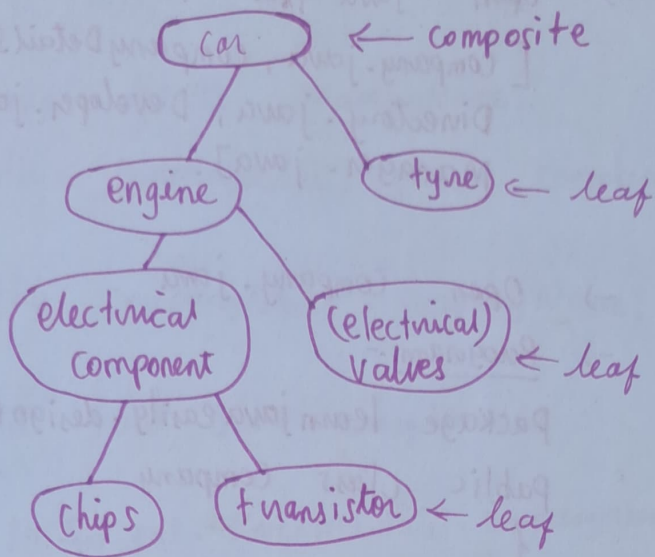
Composite Design Pattern

→ fall on structural design patterns

→ nodes → composite, leaf

complex
object



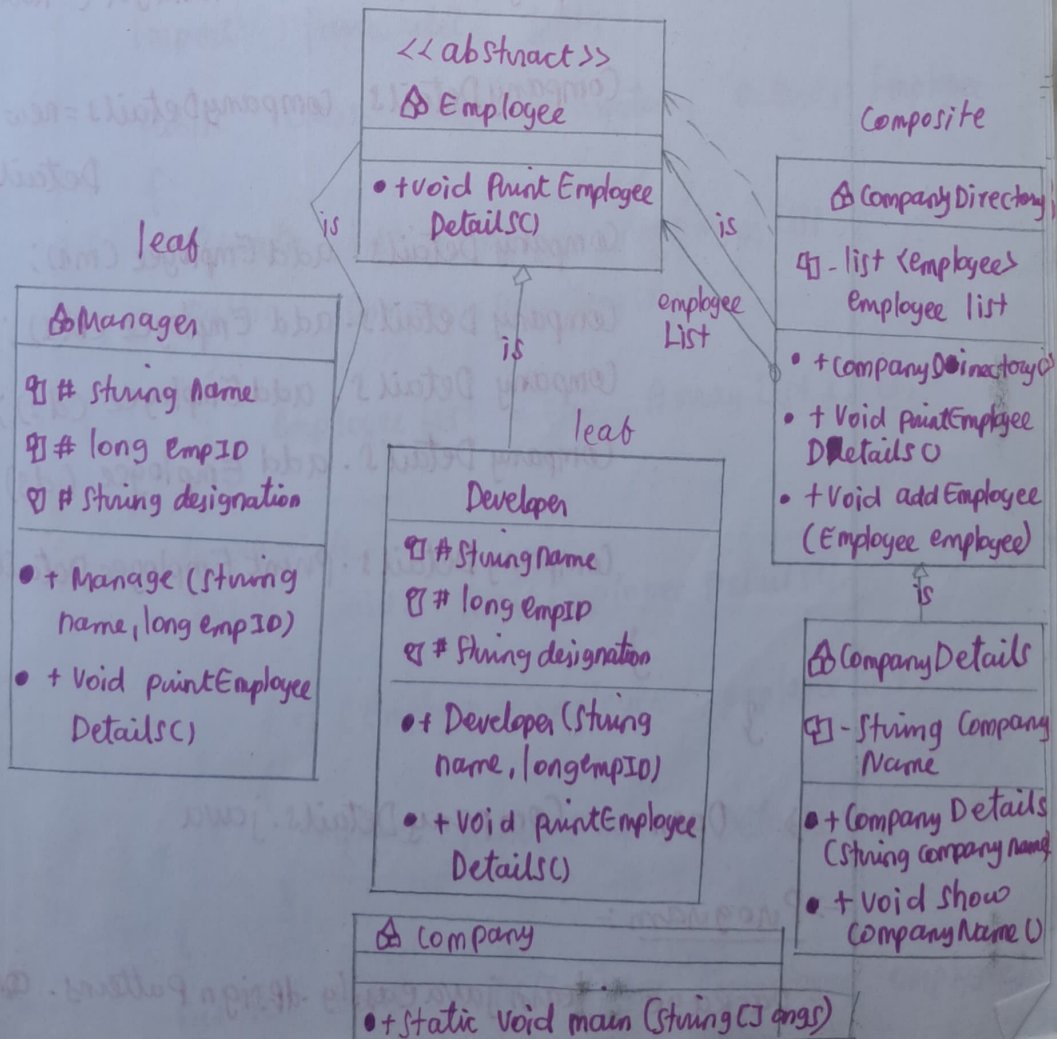


→ Operation apply on leaf and composite.
(not only on composite).

19. Composite Design Pattern Java Implementation:-

→ Open netbeans

→ Use case Diagram:-



→ open java file then create java file.

[Company.java, CompanyDetails.java, CompanyDirectory.java, Developer.java, Employee.java, Manager.java].

→ Open Company.java

→ Program :-

Package learn java easily . design Patterns . composite ;

public class Company

{

public static void main (String [] args)

{

Employee d1 = new Developer ("Lenin", 10);

Employee d2 = new Developer ("Chenhu", 11);

Employee d3 = new Developer ("Raja", 12);

Employee m1 = new Manager ("Rakesh", 1);

CompanyDetails companyDetails = new Company

Details ("xyz");

CompanyDetails.addEmployee (m1);

CompanyDetails.addEmployee (d1);

CompanyDetails.addEmployee (d2);

CompanyDetails.addEmployee (d3);

CompanyDetails.printEmployeeDetails();

}

}

→ Open CompanyDetails.java

→ Program :-

package learn java easily . design Patterns . composite ;


```

public class CompanyDetails extends CompanyDirectory
{
    private String companyName;

    public CompanyDetails (String companyName)
    {
        this. companyName = companyName;
    }

    public void showCompanyName()
    {
        System.out. println (this. companyName);
    }
}

```

→ Open CompanyDirectory. java

→ Program :-

```

package learnjavaeasily. designPatterns. Composite;

```

```

import java. util. ArrayList;

```

```

import java. util. List;

```

```

public class CompanyDirectory extends Employee

```

```

{
    private List < Employee > employeeList;

```

```

    public CompanyDirectory ()

```

```

    {
        employeeList = new ArrayList <> ();
    }

```

```

    @Override

```

```

    public void PrintEmployeeDetails ()

```

```

    {
        for (Employee employee : employeeList)

```

```

        {
            employee. PrintEmployeeDetails ();

```

```

        }
    }

    public void addEmployee (Employee employee)

```



```

    {
        employeeList.add(employee);
    }
}

```

→ Open Developer.java

→ Program :-

```

Package learnjavaeasily.designpatterns.compos
public class Developer extends Employee
{
    protected String name;
    protected long empID;
    protected String designation;
}

```

```

public Developer (String name, long empID)
{
    this.name = name;
    this.empID = empID;
}

```

@Override

```

public void PrintEmployeeDetailSC()
{
    System.out.println("Name:" + name + " empID:" +
        empID + " Designation: Developer");
}
}

```

→ Open Employee.java

→ Program :-

```

Package learnjavaeasily.designpatterns.compos
public abstract class Employee
{
}

```

Public abstract void PrintEmployeeDetails();

→ Open Manager.java

→ Program:-

package learn java easily. design patterns. composite;

public class Manager extends Employee

{

protected String name;

protected long empID;

protected String designation;

public Manager (String name, long empID)

{

this.name = name;

this.empID = empID;

@Override

public void PrintEmployeeDetails()

{

System.out.println("Name : " + name + "empID : " + empID + "Designation : Manager");

}

}

→ Output for (Company.java)

Name: Rakesh empID : 9 Designation: Manager

Name: Lenin empID : 10 Designation: Developer

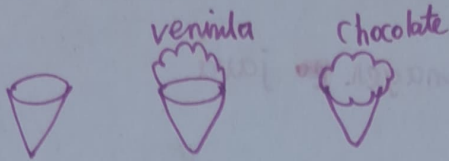
Name: Chenchu empID : 11 Designation: Developer

Name: Raja empID : 12 Designation: Developer

20. Decorator Design Pattern:-

Decorator Design Pattern

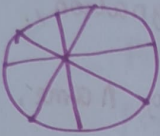
- fall on Structural design pattern
- Additional responsibilities of run time



→ base
cone



→ base
Object



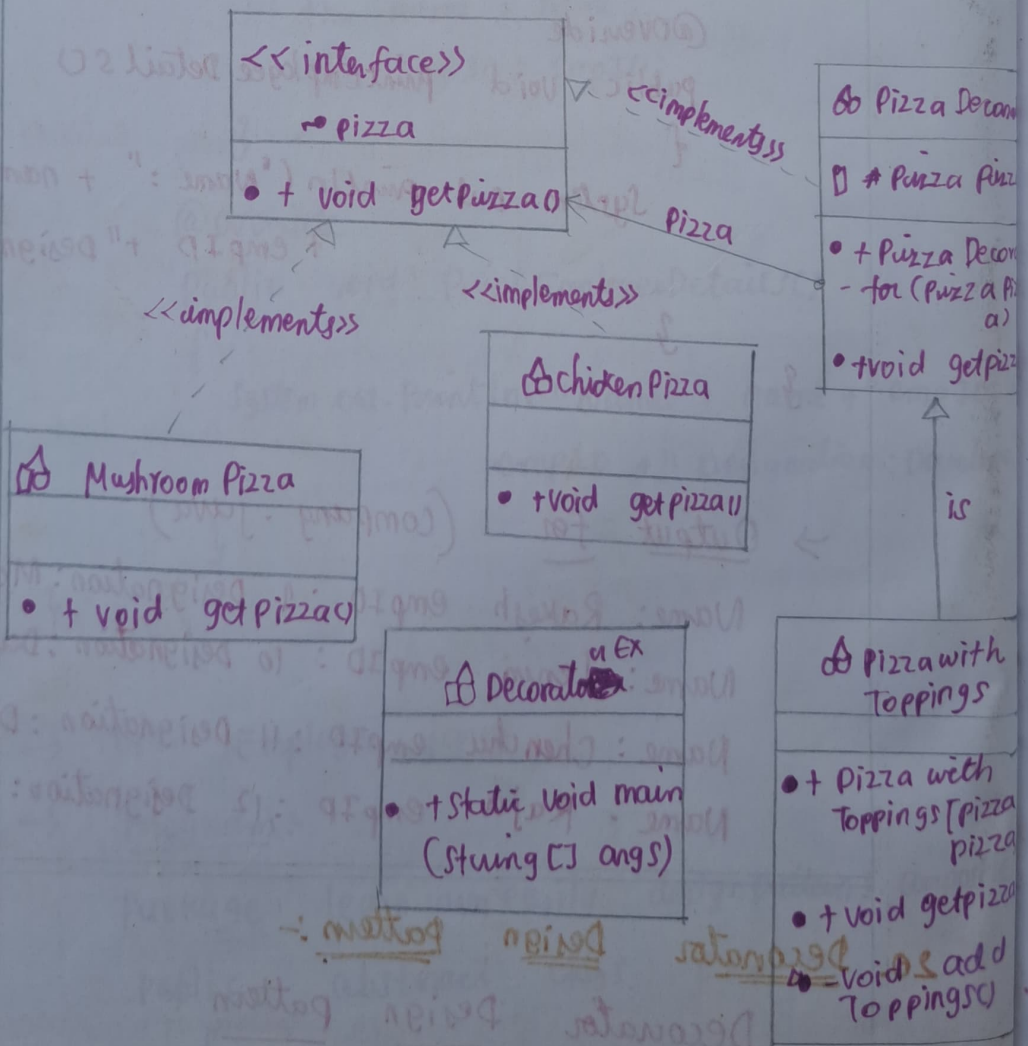
plain pizza

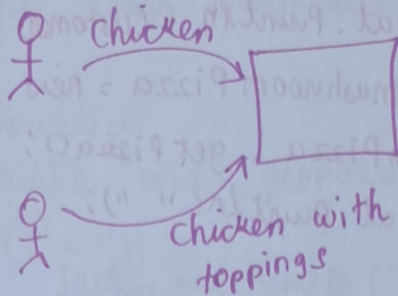


Chicken pizza



21. Decorator Design Pattern Java Program :-





- Open netbeans

- Then create java file [chickenPizza.java, DecoratorEx.java, MashnoomPizza.java, Pizza.java, PizzaDecorator.java, PizzaWithToppings.java].

- Then open chickenPizza.java

→ Program:-

```

package learnjavaeasily.designPatterns.decorator;

public class ChickenPizza implements Pizza {
    @Override
    public void getPizza() {
        System.out.println("chicken Pizza");
    }
}
  
```

→ Open DecoratorEx.java

→ Program:-

```

package learnjavaeasily.designPatterns.decorator;

public class DecoratorEx {
    System.out.println("Customer 1");
    Pizza chickenPizza = new ChickenPizza();
    ChickenPizza.getPizza();
    System.out.println("");
}
  
```

```

System.out.println ("Customer 2");
Pizza mushroom Pizza = new Mushroom Pizza();
mushroom Pizza . get Pizza();
System.out.println (" ");

```

```

System.out.println ("Customer 3");
Pizza chicken PizzaWith Toppings = new Pizza
With Toppings (new Chicken Pizza
Chicken PizzaWith Toppings . get Pizza();
System.out.println (" ");

```

```

System.out.println ("Customer 4");
Pizza mushroom Pizza With Toppings = new
Pizza With Toppings (new Mushroom
Pizza());

```

```

System.out.println (" ");
}
}

```

~~Mushroom Pizza . java~~

→ Output - Customer 1
 Chicken Pizza
 Customer 2
 Mushroom Pizza
 Customer 3
 chicken Pizza with toppings
 Customer 4
 Mushroom Pizza with toppings.

→ Open Mushroom Pizza . java

→ Program:-
 package learn java easily . design patterns
 decorator;

public class MushroomPizza implements Pizza

{

@Override

public void getPizza()

{

System.out.print("Mushroom Pizza");

}

}

→ Open pizza.java

→ Program:-

Package learnjava easily . design Patterns . decorator;

public interface Pizza

{

public void getPizza();

}

→ Open PizzaDecorator.java

→ Program:-

Package learnjava easily . design Patterns . decorator;

public abstract class PizzaDecorator implements
Pizza

{

protected Pizza pizza;

public PizzaDecorator (Pizza pizza)

{

this.pizza = pizza;

}

@Override

public void getPizza()

{

this.pizza . getPizza();

}

}

→ Open pizza with Toppings.java

→ Program :-

package learnjavaeasily . design patterns . decorator

public class pizzaWithToppings extends
pizzaDecorator

{

public pizzaWithToppings (pizza pizza)

{

super (pizza);

}

@Override

public void getPizza()

{

pizza . getPizza();

addToppings();

}

private void addToppings()

{

System.out.println ("with toppings");

}

}

22. Facade Design Pattern :-

→ Facade Design Pattern :-

→ single class represents entire subsystem

→ simple interface to a complex system

→ french "facade" → frontage

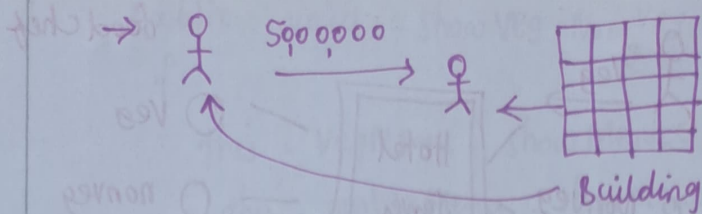
→ 10,000 withdraw → Account No. Valid

→ funds

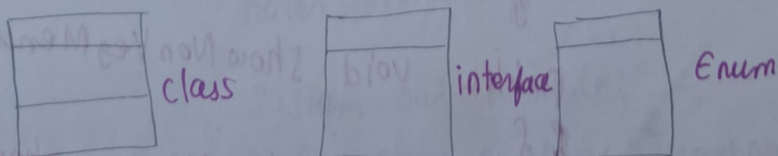
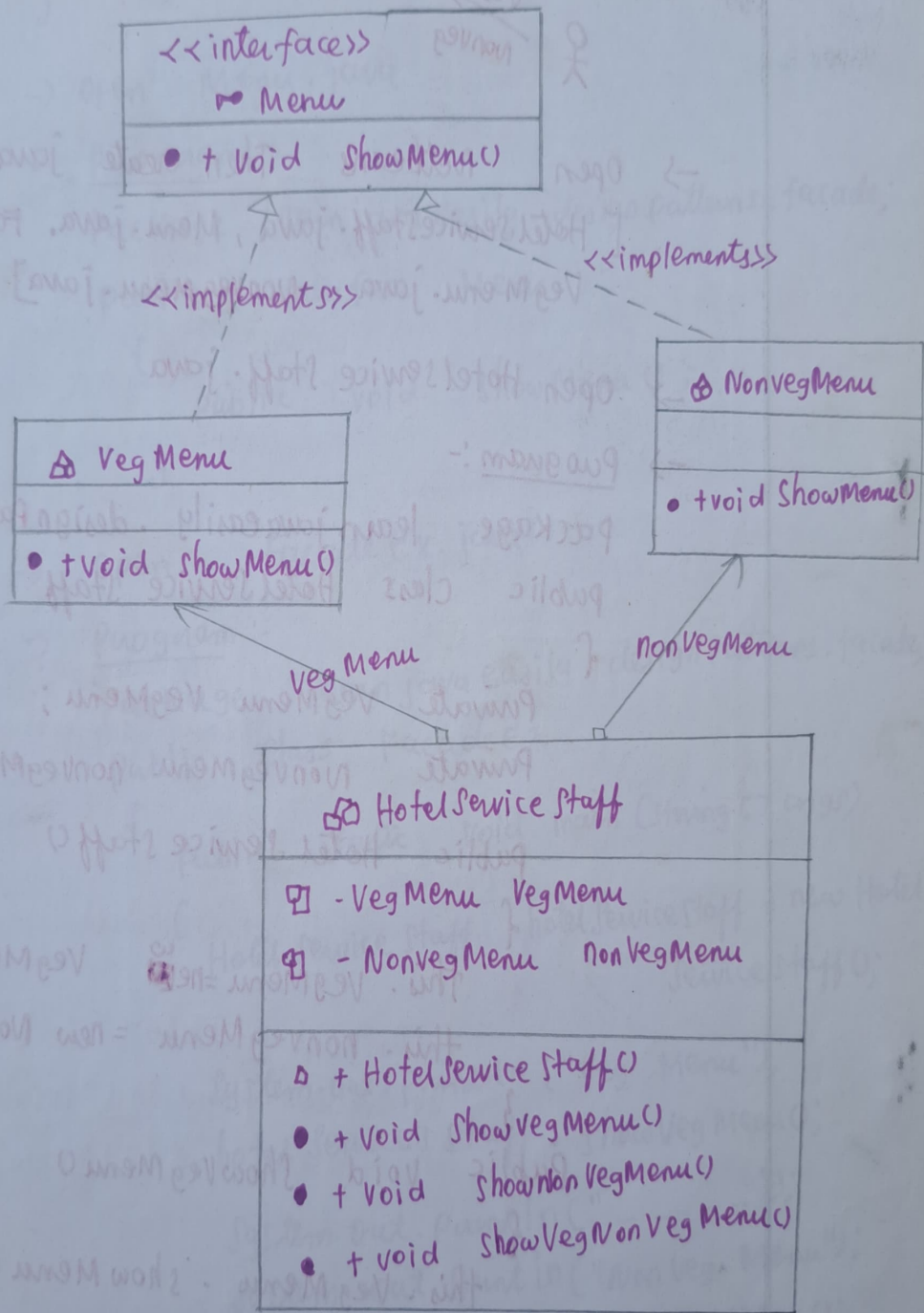
→ security code



ATM



23. Facade Design Pattern In java :-



Relation :-

→ is

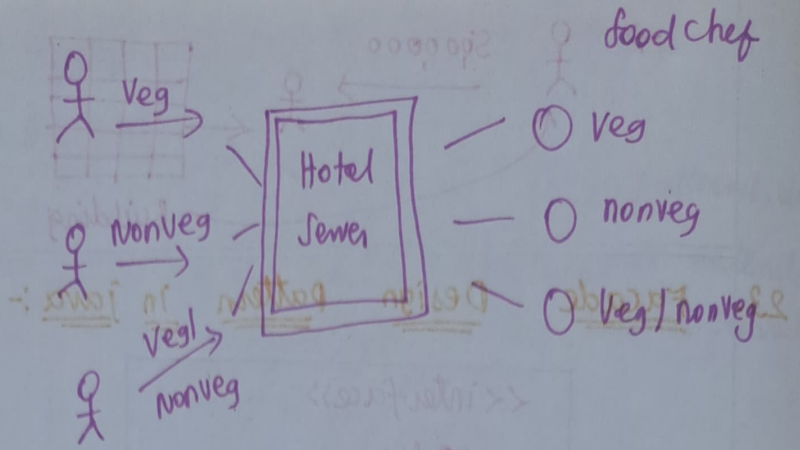
---> implements

→ Has

◊→ Has (agg)

◄→ Has (comp)

---> use



→ Open Netbeans Then create java file
 [HotelServiceStaff.java, Menu.java, Facade.java,
 VegMenu.java, NonvegMenu.java]

→ Open HotelServiceStaff.java

→ Package:-

package learnjavaeasily.designPatterns.facade;

public class HotelServiceStaff

{

private VegMenu vegMenu;

private NonvegMenu nonvegMenu;

public HotelServiceStaff()

{

this.vegMenu = new VegMenu();

this.nonvegMenu = new NonvegMenu();

}

public void showVegMenu()

{

this.vegMenu.showMenu();

}

public void showNonVegMenu();

{

this.nonVegMenu.showMenu();

}


```

    public void showVeg Non Veg Menu()
    {
        this.vegMenu.showMenu();
        this.nonvegMenu.showMenu();
    }
}

```

→ open Menu.java

→ Program :-

```

package learnjavaeasily.designpatterns.facade;

public interface Menu
{
    public void showMenu();
}

```

→ open FacadeEx.java

→ Program :-

```

package learnjavaeasily.designpatterns.facade;

public class FacadeEx
{
    public static void main (String[] args)
    {
        HotelServiceStaff hotelServiceStaff = new HotelServiceStaff();
    }
}

```

```

        System.out.println("Veg Menu");
        hotelServiceStaff.showVegMenu();

        System.out.println("-----");
        System.out.println("Non Veg Menu");
        hotelServiceStaff.showNonVegMenu();

        System.out.println("-----");
        System.out.println("Veg & Non Veg Menu");
        hotelServiceStaff.showVegNonVegMenu();
    }
}

```

→ Output - Veg Menu

Idly

Dosa

Vada

Non Veg Menu

Chicken Baniyani

Tandoori

Veg & Non Veg Menu

Idly

Dosa

Vada

Chicken Baniyani

Tandoori

→ Open VegMenu.java

→ Program:-

Package learnjavaeasily.designPatterns.facade;

Public class VegMenu implements Menu

{

@Override

Public void showMenu()

{

System.out.println("Idly");

System.out.println("Dosa");

System.out.println("Vada");

}

}

→ Open NonVegMenu.java

→ Program:-

Package learnjavaeasily.designPatterns.facade;

Public class NonVegMenu implements Menu

{

@Override
public void showMenu()

```
{  
    System.out.println("Chicken Biriyani");  
    System.out.println("Tandoori");  
}
```

24. Flyweight Design pattern:-

Flyweight Design pattern

→ It is used to create a large number of objects of almost similar nature.

→ Large number of objects consume large amount of memory.

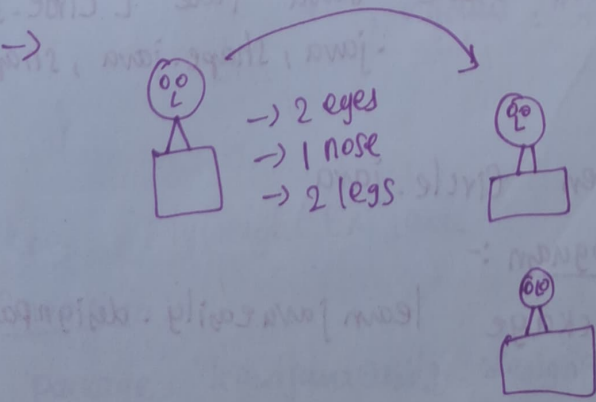
→ Flyweight design pattern provides a solution for reducing the load on memory by sharing objects.

→ It is used to reduce the number of objects created and improve performance.

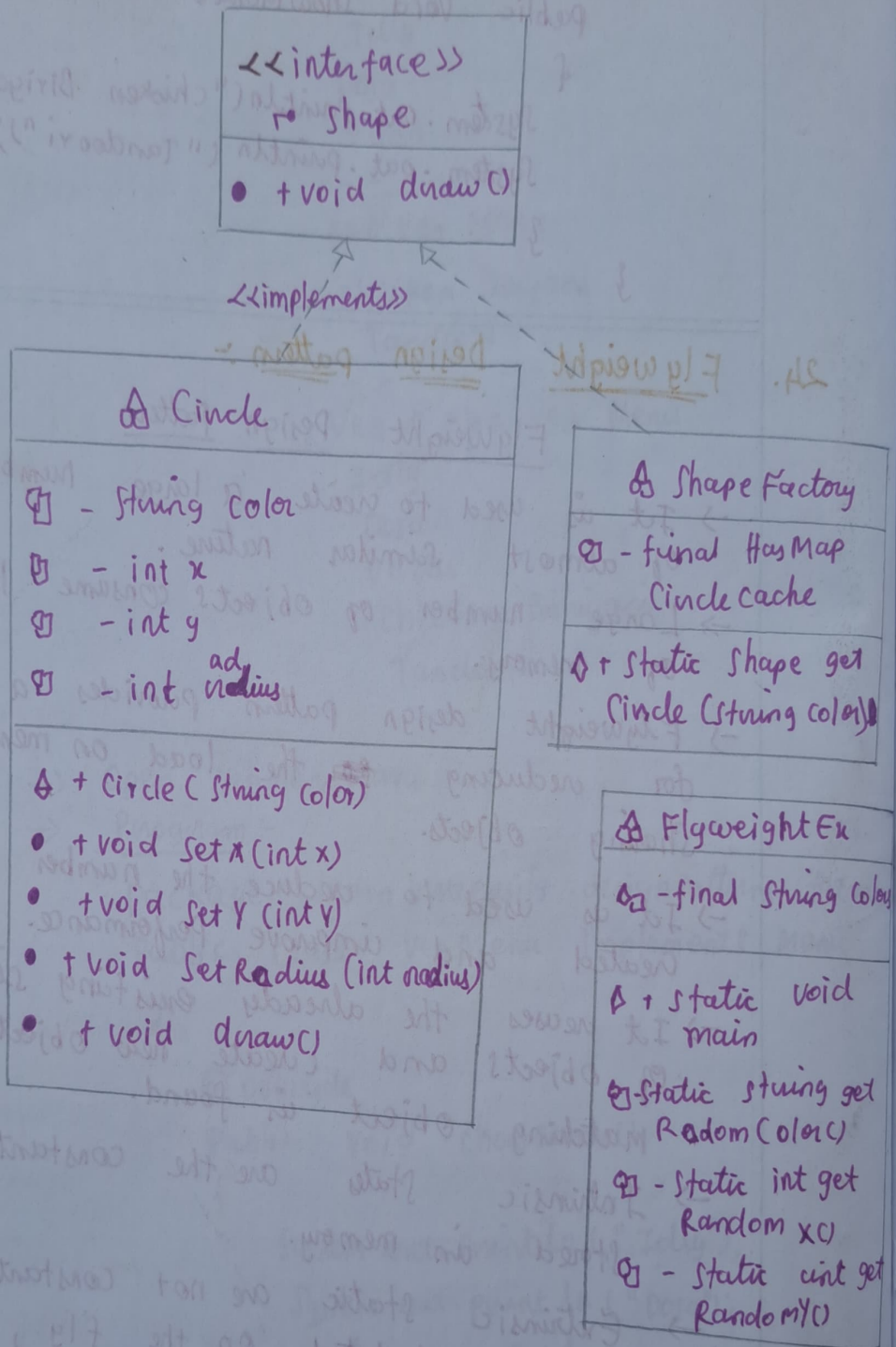
→ It reuses the already existing similar kind of objects and create new object when no matching object is found.

→ Intrinsic states are the constants and are stored in memory.

→ Extrinsic states are not constants and need to be calculated on the fly, they are not stored in memory.



25. Flyweight Design Pattern in Java:-



→ Open netbeans

→ Then create Java file [Circle.java, Flyweight.java, Shape.java, ShapeFactory.java]

→ Open Circle.java

→ Program :-

package learn java easily . design patterns . flyweight ;

Public class Circle implements shape
{

Private String color;

Private int x;

Private int y;

Private int radius;

Public Circle (String color)

{

this.color = color;

}

public void setX (int x)

{

this.x = x;

}

public void setY (int y)

{

this.y = y;

}

public void setRadius (int radius)

{

this.radius = radius;

}

@Override

public void draw()

{

System.out.println (" Circle: Draw() [
color : " + color + ", x : " + x + ", y : " + y
+ ", radius : " + radius);

}

}

- Open Flyweight Ex. java

Program :-

package learnjavaeasily.designPatterns.flyweight;

```
public class FlyweightEx
```

```
{
```

```
    private static final String colors[] =
```

```
    {
```

```
        "Red", "Green", "Blue", "White", "Black"
```

```
    };
```

```
    public static void main (String[] args)
```

```
    {
```

```
        for (int i=0; i<10; ++i)
```

```
        {
```

```
            String targetColor = getRandomColor();
```

```
            System.out.println("Target color: " + targetColor);
```

```
            Circle circle = (Circle) ShapeFactory.getCircle(targetColor);
```

```
            circle.setX(getRandomX());
```

```
            circle.setY(getRandomY());
```

```
            circle.setRadius(50);
```

```
            circle.draw();
```

```
        } }
```

```
    private static String getRandomColor()
```

```
    {
```

```
        return colors[(int) (Math.random() * colors.length)];
```

```
    }
```

```
    private static int getRandomX()
```

```
    {
```

```
        return (int) (Math.random() * 100);
```

```
    }
```

```
    private static int getRandomY()
```

```
    {
```

```
        return (int) (Math.random() * 100);
```

```
    }
```


→ Output -

Target color: Black

There is no cached object so creating Circle of Color: Black

Circle: Draw() [color: Black, x: 81, y: 80, radius: 50]

Target color: Black

Circle: Draw() [color: Black, x: 61, y: 92, radius: 50]

Target color: Red

There is no cached object so creating Circle of color: white

Circle: Draw() [color: white, x: 85, y: 86, radius: 50]

Target color: white

There is no cached object so creating circle of color: white

Circle: Draw() [color: white, x: 1, y: 36, radius: 50]

Target color: Red

Circle: Draw() [color: Red, x: 67, y: 97, radius: 50]

Target color: white

Circle: Draw() [color: white, x: 4, y: 64, radius: 50]

Target color: Black

Circle: Draw() [color: Black, x: 54, y: 71, radius: 50]

Target color: white

Circle: Draw() [color: white, x: 62, y: 57, radius: 50]

Target color: Blue

There is no cached object so creating circle of color: Blue

Circle: Draw() [color: Blue, x: 94, y: 82, radius: 50]

Target color: Green

There is no cached object so creating circle of color: Green

Circle: Draw() [color: Green, x: 14, y: 3, radius: 50]

→ Open shap.java

→ Program:-

package learnjavaeasily.designpatterns.flyweight;

public interface shape

{
public void draw();
}

→ Open shape factory. java

→ Prोग्राम :-

Package learn java easily. design patterns. flyweight
import java.util. HashMap;

Public class shape Factory

{
Private static final HashMap CircleCache;
new HashMap ();

Public static shape get circle (String
color)

{

Circle circle = (Circle) CircleCache.
get (color);

if (circle == null)

{

Circle = new Circle (color);

CircleCache. put (color, circle);

System.out.println ("There is no
Cached object so creating
Circle of color: " + color);

}

return circle;

}

}

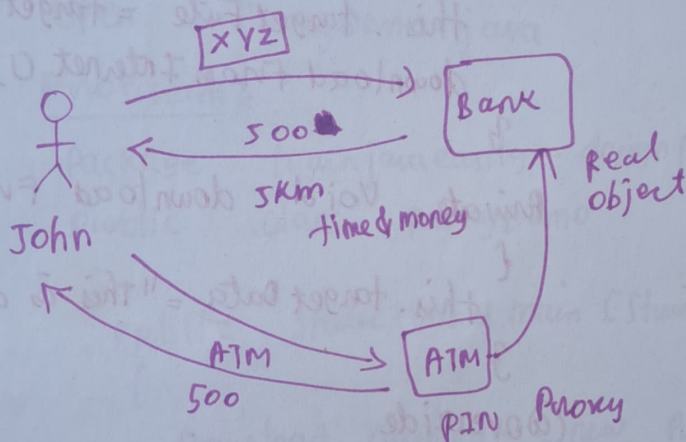
26. Proxy Design Pattern :-

→ Proxy - "in place of" or "representing" or
behalf of "

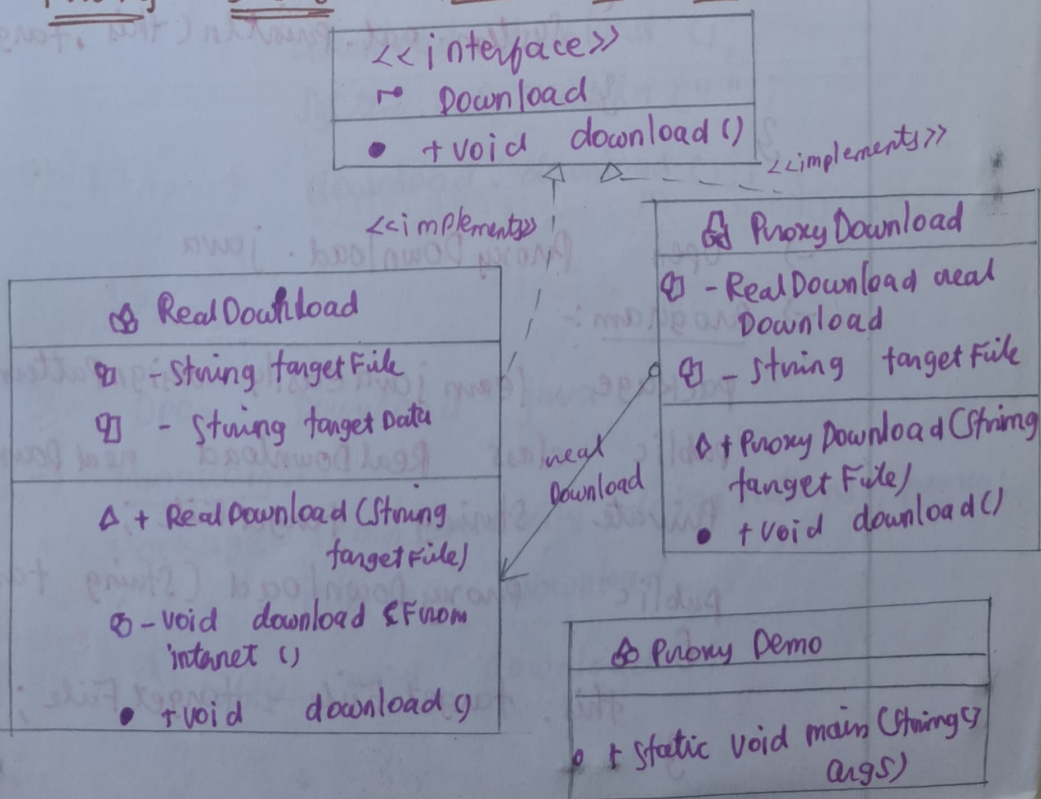
→ placeholder for another object to control the
access.

→ Proxy is the object that is being
called by the client to access the
object behind the scene.

- Proxy Pattern represents the functionality of another class.
- Virtual Proxy is a placeholder for "expensive to create" objects. The real object is only ~~provided~~ created when a client requests/ access the object.
- Remote Proxy provides a local representative for an object that resides in a different address space.
- Protective Proxy controls access to a sensitive master object. It checks whether the caller has permission and etc.



27. Proxy Design Pattern in Java :-



→ Open netbeans Then create Java file [Real Download.java, Proxy Download.java, ProxyDemo.java, Download.java]

→ Program:-

package learnjavaeasy.designpatterns.proxy;

public class RealDownload implements Download

{

private String targetFile;

private String targetData;

public RealDownload (String targetFile)

{

this.targetFile = targetFile;

downloadFromInternet();

}

private void downloadFromInternet()

{

this.targetData = "This is a test data";

}

@Override

public void download()

{

System.out.println(this.targetData);

}

}

→ Open ProxyDownload.java

→ Program:-

package learnjavaeasy.designpatterns.proxy;

public class RealDownload implements Download;

private String targetFile;

public ProxyDownload (String targetFile)

{

this.targetFile = targetFile;

@ Override: Method Design for Visitor .82

```
public void download ()  
{  
    if (real Download == null)  
    {  
        real Download = new Real Download (target  
            File);  
    }  
    real Download . download ();  
}  
}
```

→ Open ProxyDemo . java

→ Program:-

```
Package learnjavaeasily . designPatterns . Proxy;  
public class ProxyDemo
```

```
{  
    public static void main (String [] args)  
    {
```

```
        Download download = new ProxyDownload  
            ("xyz . movie");
```

```
        download . download ();
```

```
        System . out . println ("");
```

```
        download . download ();  
    }  
}
```

→ Open Download . java

→ Program:-

```
Package learnjavaeasily . designPatterns . Proxy;
```

```
public interface Download
```

```
{  
    public void download ();
```

```
}
```


28. Visitor Design Pattern :-

- It changes the executing algorithm of an element object.
- Execution logic varies as the visitor varies.
- Element object has to accept visitor object so that visitor object handles the operation on the element object.
- Example : Supermarket products Price with tax example.

	Jan	Feb	MAR
food	5%	10%	3%
toys	10%	20%	10%
Dress	3%	10%	15%
utensils	10%	8%	18%

Shopping Mall

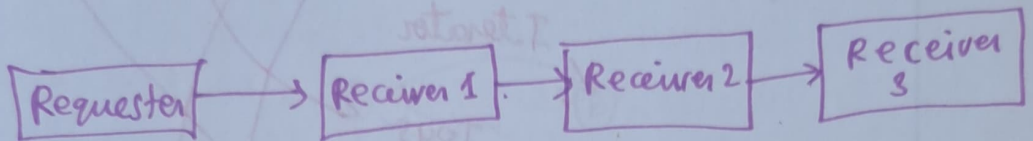
Visitor object

29. Chain of responsibility Design Pattern :-

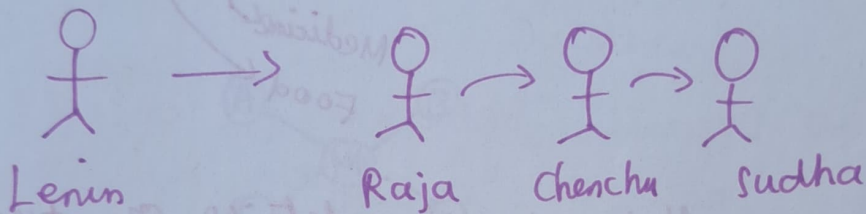
- Avoid coupling the sender of a request to its receiver by giving more than one receiver object a chance to handle the request.
- chain the receiving objects and pass the request along the chain until an object handles it.
- It creates a chain of receiver objects of a request.

→ Each receiver contains reference to another receiver, if one receiver cannot handle the request then it will pass to the next receiver.

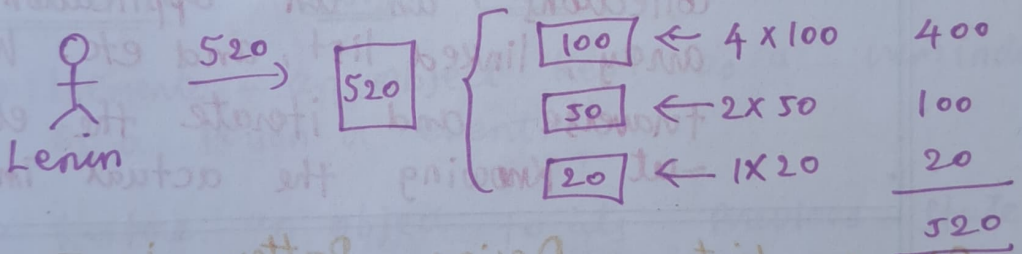
→ (one receiver handles a request in the chain) or (one or more receivers in the chain handles a request).



One receiver



One or more Receivers



30. Iterator Design Pattern:-

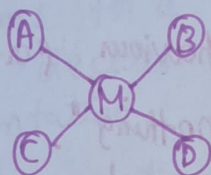
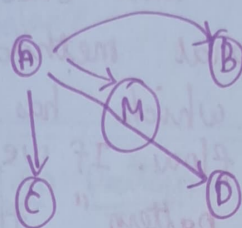
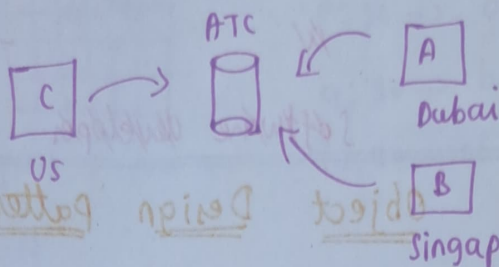
→ Access the elements of a collection objects in sequential manner without any need to know its underlying representation.

→ Collection can be a List, Set, Array and it can be anything.

→ In OOPS, Iterator Pattern is a design pattern in which an iterator is used to traverse a container and access the container's elements.

→ Reduce the coupling between the objects.

Chennai Airport



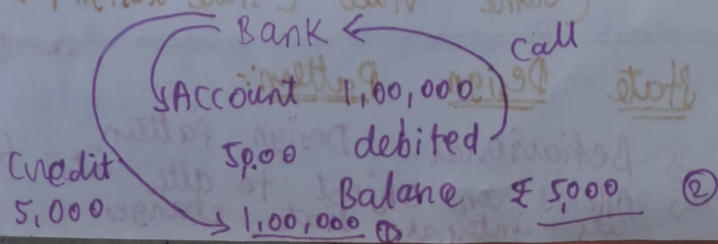
32. Memento Design Pattern :-

→ Memento - an object kept as a reminder of a person or event.

→ Restore an object to its previous state.
Example : "undo" or "rollback".

→ Basic object that is stored in different states.

→ Caretaker : Holds an ArrayList that contains all previous versions of the Memento. It can store and retrieve stored Mementos.



Rakesh

→ ①

Xs

→ ②

✓

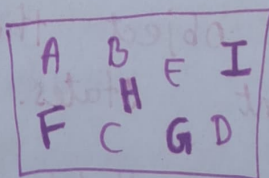
→ ③ (undo)

Software Developer

→ 4

33. Null Object Design Pattern :-

- Replace "null" check.
- Traditional method: null check for every value which has some limitations in code flow. If we follow "null object design pattern" then we can overcome that hurdle.
- Default behaviour for objects which has "null" or "nothing".
- Null object class is the default implementation.
- It would be very useful if we want some special flow for something which has nothing on null value.
- Example: Spam Classification.



ABCDE

34. Observer Design Pattern :-

(Same video (State design pattern))

35. State Design Pattern :-

- Behavioral Design Pattern
- Allows an object to alter its behaviour its internal state changes.

ATM

Human

1) Card not inserted



8:30 → office travel

9:30 → break fast

10:00 - 1:00 → work

2) Card insert

→ PIN

→ withdraw

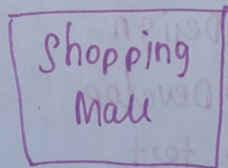
→ check balance

→

36. Strategy Design Pattern:-

→ It is used when there are multiple algorithms for a specific task and the client decides the actual and needed implementation.

→ Multiple solutions for a particular problem and client decides at the run time based on various criteria.



↓
10,000 Rs purchase

↓
Bill counter

Debit

Credit

Cash

37. Template Method Design Pattern:-

→ It defines a sequence of steps of an algorithm.

→ Subclasses can override the steps but it won't allow to alter the sequence of

Steps.

→ House building : basement, pillars, side walls, interior - concrete and wooden house.

→ Software development : Analysis, design, develop and test.

→ basement

→ Pillars

→ Side walls

→ interior

Cement
House

wooden
House

Software

A

→ Analyze

→ Design

→ Develop

→ test

B

→ Analyze

→ Design

→ Develop

→ test